

```
setwd("/Users/apple/Documents/UOW/INFO911 Data Mining and Knowledge  
Discovery/assignment2")
```

```
library(tree)  
library(rpart)  
library(rpart.plot)  
library(randomForest)
```

```
###
```

```
# Split the dataset into 50% training set and 50% test set and only include the data with  
known ratings
```

```
credit_data <- read.csv('creditworthiness.csv')  
credit_data_known <- credit_data[credit_data$credit.rating != 0, ]
```

```
# Convert all columns to factors using a loop  
for (colname in names(credit_data_known)) {  
  credit_data_known[[colname]] <- as.factor(credit_data_known[[colname]])  
}
```

```
# Randomly select half of the samples
```

```
set.seed(123)  
set=sample(1:nrow(credit_data_known),nrow(credit_data_known)*0.8, replace=FALSE)  
# Create the train and test sets  
credit.train=credit_data_known[set,]  
credit.test=credit_data_known[-set,]
```

```
###
```

```
# Report the resulting tree
```

```
credit.treetrn = rpart(credit.rating ~ ., data = credit.train, parms =  
  list(split = "information"))
```

```
n= 1569
```

```
node), split, n, loss, yval, (yprob)
```

```
  * denotes terminal node
```

```
1) root 1569 782 2 (0.24665392 0.50159337 0.25175271)  
 2) functionary=1 471 228 1 (0.51592357 0.32059448 0.16348195)  
   4) FI3O.credit.score=1 451 209 1 (0.53658537 0.33037694 0.13303769)  
      8) savings.on.other.accounts=1,2,3,5,6 441 199 1 (0.54875283 0.31519274  
0.13605442) *  
        9) savings.on.other.accounts=4 10 0 2 (0.00000000 1.00000000 0.00000000) *  
   5) FI3O.credit.score=0 20 3 3 (0.05000000 0.10000000 0.85000000) *  
 3) functionary=0 1098 462 2 (0.13114754 0.57923497 0.28961749)  
   6) FI3O.credit.score=1 1023 402 2 (0.14076246 0.60703812 0.25219941)  
      12) re.balanced..paid.back..a.recently.overdrawn.current.acount=1 946 346 2  
(0.14904863 0.63424947 0.21670190) *  
        13) re.balanced..paid.back..a.recently.overdrawn.current.acount=0 77 24 3 (0.03896104  
0.27272727 0.68831169) *  
   7) FI3O.credit.score=0 75 15 3 (0.00000000 0.20000000 0.80000000) *
```

```

rpart.plot(credit.treetrn, extra=101, type=3)
# confusion matrix
train.cm <- table(credit.train$credit.rating, predict(credit.treetrn, newdata=credit.train,
type="class"))
1  2  3
1 242 141  4
2 139 610 38
3  60 205 130
# calculate accuracy
train.accuracy <- sum(diag(train.cm))/sum(train.cm)
0.6258764

```

###

Based on this output, predict the credit rating of a hypothetical "median" customer, i.e., one with the attributes listed in Table 1, showing the steps involved.

Create a data frame for the median customer

```

median_customer <- data.frame(
  functionary = 0,
  `re-balanced (paid back) a recently overdrawn current account` = 1,
  `FI3O credit score` = 1,
  gender = 0,
  `0. accounts at other banks` = 3,
  `credit refused in past?` = 0,
  `years employed` = 3,
  `savings on other accounts` = 3,
  `self employed?` = 0,
  `max. account balance 12 months ago` = 3,
  `min. account balance 12 months ago` = 3,
  `avrg. account balance 12 months ago` = 3,
  `max. account balance 11 months ago` = 3,
  `min. account balance 11 months ago` = 3,
  `avrg. account balance 11 months ago` = 3,
  `max. account balance 10 months ago` = 3,
  `min. account balance 10 months ago` = 3,
  `avrg. account balance 10 months ago` = 3,
  `max. account balance 9 months ago` = 3,
  `min. account balance 9 months ago` = 3,
  `avrg. account balance 9 months ago` = 3,
  `max. account balance 8 months ago` = 3,
  `min. account balance 8 months ago` = 3,
  `avrg. account balance 8 months ago` = 3,
  `max. account balance 7 months ago` = 3,
  `min. account balance 7 months ago` = 3,
  `avrg. account balance 7 months ago` = 3,
  `max. account balance 6 months ago` = 3,
  `min. account balance 6 months ago` = 3,
  `avrg. account balance 6 months ago` = 3,
  `max. account balance 5 months ago` = 3,

```

```

`min. account balance 5 months ago` = 3,
`avrg. account balance 5 months ago` = 3,
`max. account balance 4 months ago` = 3,
`min. account balance 4 months ago` = 3,
`avrg. account balance 4 months ago` = 3,
`max. account balance 3 months ago` = 3,
`min. account balance 3 months ago` = 3,
`avrg. account balance 3 months ago` = 3,
`max. account balance 2 months ago` = 3,
`min. account balance 2 months ago` = 3,
`avrg. account balance 2 months ago` = 3,
`max. account balance 1 months ago` = 3,
`min. account balance 1 months ago` = 3,
`avrg. account balance 1 months ago` = 3
)
# Convert to factors
for (col in names(median_customer)) {
  median_customer[[col]] <- as.factor(median_customer[[col]])
}
# Match column names with the training dataset
names(median_customer) <- names(credit.train)[-ncol(credit.train)]
# Predict the credit rating
predicted_rating <- predict(credit.treetrn, median_customer, type = "class")
print(paste("Predicted credit rating for median customer:", predicted_rating))
"Predicted credit rating for median customer: 2"

####
# Produce the confusion matrix for predicting the credit rating from this tree on the test set,
and also report the overall accuracy rate.
# confusion matrix
test.cm <- table(credit.test$credit.rating, predict(credit.treetrn, newdata=credit.test,
type='class'))
  1  2  3
1 50 41  5
2 30 140 13
3 19  64 31
# calculate accuracy
test.accuracy <- sum(diag(test.cm))/sum(test.cm)
0.562341

####
# What is the numerical value of the gain in entropy corresponding to the first split at the top
of the tree?
# (Use logarithms to base 2, and show the details of the calculation rather than just providing
a final answer.)
#first split:
#1) root 981 488 2 (0.50254842 0.24974516 0.24770642)
#2) functionary=0 681 290 2 (0.57415565 0.13509545 0.29074890)

```

```
#3) functionary=1 300 147 1 (0.34000000 0.51000000 0.15000000)
#Class probabilities at root:
#Class 2: 0.50254842 | Class 1: 0.24974516 | Class 3: 0.24770642
#Hroot = -sum(p*log2(p)) = 1.4974
Hroot = -(0.5025*log2(0.5025) + 0.2497*log2(0.2497) + 0.2477*log2(0.2477))
```

```
#Class probabilities at node 2:
#Class 2: 0.57415565 | Class 1: 0.13509545 | Class 3: 0.29074890
#H2 = -sum(p*log2(p)) = 1.3679
H2 = -(0.5742*log2(0.5742) + 0.1351*log2(0.1351) + 0.2907*log2(0.2907))
```

```
#Class probabilities at node 3:
#Class 2: 0.34000000 | Class 1: 0.51000000 | Class 3: 0.15000000
#H3 = -sum(p*log2(p)) = 1.4351
H3 = -(0.34*log2(0.34) + 0.51*log2(0.51) + 0.15*log2(0.15))
```

```
#Weighted entropy after split
#Hsplit = 1.3884
Hsplit = H2*(681/981) + H3*(300/981)
```

```
#Information gain
#Gain = 0.109
Gain = Hroot - Hsplit
```

```
####
# Fit a random forest model to the training set to try to improve prediction. Report the R
output.
# Set a seed for reproducibility
set.seed(123)
# Determine number of predictors (p)
p <- ncol(credit.train) - 1
# Try a range of hyperparameters
mtry.values <- c(floor(sqrt(p)), floor(p/2), p)
ntree.values <- c(100, 250, 300, 350, 450, 500, 550, 600, 700, 800, 900)
nodesize.values <- c(1, 5, 10)
# To store results
results <- data.frame()
# Grid search over all combinations
for (m in mtry.values) {
  for (n in ntree.values) {
    for (ns in nodesize.values) {
      credit.rf <- randomForest(
        credit.rating ~ .,
        data = credit.train,
        ntree = n,
        mtry = m,
        nodesize = ns,
        importance = TRUE
```

```

    )
    # Out-of-bag (OOB) accuracy
    oob.accuracy <- 1 -
credit.rf$confusion[,"class.error"][which.max(table(credit.train$credit.rating))]
    # Accuracy on test set
    rf.pred <- predict(credit.rf, newdata = credit.test)
    rf.cm <- table(credit.test$credit.rating, rf.pred)
    rf.accuracy <- sum(diag(rf.cm)) / sum(rf.cm)
    # Store results
    results <- rbind(results, data.frame(
      mtry = m,
      ntree = n,
      nodesize = ns,
      oob.accuracy = oob.accuracy,
      rf.accuracy = rf.accuracy
    ))
  }
}
}
# View best result
best <- results[which.max(results$rf.accuracy), ]
print(best)
  mtry ntree nodesize oob.accuracy rf.accuracy
215   6   500       1  0.8297332  0.5801527

```

```

####
library(e1071)
# Fit SVM model with default settings
set.seed(123)
credit.svm <- svm(credit.rating ~ ., data = credit.train, probability = TRUE)
Parameters:
  SVM-Type: C-classification
  SVM-Kernel: radial
  cost: 1

```

Number of Support Vectors: 1451

```

# Make predictions on test set
svm.pred <- predict(credit.svm, newdata = credit.test)

```

Predict the credit rating of a hypothetical “median” customer, i.e., one with the attributes listed in Table 1. Report decision values and discuss it.

```

svm.median.prediction <- predict(credit.svm, median_customer, type = "class")
print(paste("SVM predicted credit rating for median customer:", svm.median.prediction))
"SVM predicted credit rating for median customer: 2"

```

```

# Get decision values

```

```
svm.median.decisionvalues <- predict(credit.svm, median_customer, decision.values =
TRUE)
decision.values <- attr(svm.median.decisionvalues, "decision.values")
      3/2      3/1      2/1
1 -0.8859228 0.4733287 0.909846
3/2 = -0.8859228 (Class 3 vs Class 2): The strong negative value indicates the customer is
firmly classified as Class 2 rather than Class 3
3/1 = 0.4733287 (Class 3 vs Class 1): The positive value suggests a slight preference for
Class 3 over Class 1
2/1 = 0.909846 (Class 2 vs Class 1): The strong positive value strongly favors Class 2 over
Class 1
Class 2 wins against both Class 3 (negative 3/2) and Class 1 (positive 2/1). This creates a
clear voting majority for Class 2
```

Produce the confusion matrix for predicting the credit rating from this SVM on the test set, and also report the overall accuracy rate.

Create confusion matrix

```
svm.cm <- table(truth = credit.test$credit.rating, prediction = svm.pred)
      prediction
truth  1  2  3
      1 50 46  0
      2 34 149  0
      3 22  92  0
```

Calculate accuracy

```
svm.accuracy <- sum(diag(svm.cm)) / sum(svm.cm)
0.5063613
```

Automatically or manually tune the SVM to improve prediction over that found in 3b. Report the resulting SVM settings and the resulting confusion matrix for predicting the test set. Discuss your results.

```
set.seed(123)
```

Define more conservative tuning parameters

```
tuned.model <- tune.svm(
  credit.rating ~ .,
  data = credit.train,
  kernel = "radial",
  gamma = c(0.001, 0.01, 0.1),
  cost = c(0.1, 1, 10),
  tunecontrol = tune.control(cross = 5)
)
print(tuned.model$best.parameters)
      gamma cost
5 0.01 0.1
```

```
credit.svm.tuned <- tuned.model$best.model
Parameters:
```

SVM-Type: C-classification
SVM-Kernel: radial
cost: 0.1

Number of Support Vectors: 1301

```
# Make predictions on scaled test set
svm.tuned.pred <- predict(credit.svm.tuned, newdata = credit.test)
# Create confusion matrix
svm.tuned.cm <- table(truth = credit.test$credit.rating, prediction = svm.tuned.pred)
  prediction
truth 1  2  3
  1 96  0  0
  2  0 183  0
  3  0 105  9

# Calculate accuracy
svm.tuned.accuracy <- sum(diag(svm.tuned.cm)) / sum(svm.tuned.cm)
print(paste("Tuned SVM Accuracy:", round(svm.tuned.accuracy * 100, 2), "%"))
"Tuned SVM Accuracy: 73.28 %"

# Compare with original accuracy
print(paste("Original SVM Accuracy:", round(svm.accuracy * 100, 2), "%"))
"Original SVM Accuracy: 50.64 %"
print(paste("Improvement:", round((svm.tuned.accuracy - svm.accuracy) * 100, 2), "%"))
"Improvement: 22.65 %"
```

```
####
# Fit the Naive Bayes model to predict the credit ratings of customers using all of the other
variables in the dataset.
set.seed(123)
credit.nb <- naiveBayes(credit.rating ~ ., data = credit.train)
```

```
# Predict the credit rating of a hypothetical "median" customer
# Predict credit rating for median customer using Naive Bayes
nb.median.prediction <- predict(credit.nb, newdata = median_customer)
print(paste("Predicted credit rating for median customer using Naive Bayes: ",
nb.median.prediction))
"Predicted credit rating for median customer using Naive Bayes: 2"
```

```
# Get class probabilities for median customer
nb.median.probs <- predict(credit.nb, newdata = median_customer, type = "raw")
  1      2      3
[1,] 0.1950637 0.6531987 0.1517376
```

Reproduce the first 20 or so lines of the R output for the Naive Bayes fit, and use them to explain the steps involved in making this prediction.

1. Prior Probabilities

These represent the baseline distribution of credit ratings in the training data:

Class 1: 24.67% (low risk)

Class 2: 50.16% (medium risk) - most common rating

Class 3: 25.18% (high risk)

2. Conditional Probabilities

These show $P(\text{feature}|\text{class})$ - the probability of each feature value given each class.

For the median customer with attributes:

functionary = 0

Y 0

1 0.3720930

2 0.8081321

3 0.8050633

re-balanced account = 1

Y 1

1 0.992248062

2 0.965692503

3 0.832911392

FI3O credit score = 1

Y 1

1 0.997416021

2 0.978398983

3 0.805063291

gender = 0

Y 0

1 0.4521964

2 0.4853875

3 0.5341772

3. Prediction Calculation

3.1. Multiply conditional probabilities for each class:

$P(\text{features}|\text{Class 1}) = 0.372 \times 0.992 \times 0.997 \times 0.452 = 0.1663$

$P(\text{features}|\text{Class 2}) = 0.808 \times 0.966 \times 0.978 \times 0.485 = 0.3702$

$P(\text{features}|\text{Class 3}) = 0.805 \times 0.833 \times 0.805 \times 0.534 = 0.2883$

3.2. Multiply by prior probabilities:

$P(\text{Class 1}|\text{features}) \propto 0.1663 \times 0.2467 = 0.0410$

$P(\text{Class 2}|\text{features}) \propto 0.3702 \times 0.5016 = 0.1857$

$P(\text{Class 3}|\text{features}) \propto 0.2883 \times 0.2518 = 0.0726$

3.3. Normalize to get probabilities:

Sum = $0.0410 + 0.1857 + 0.0726 = 0.2993$

$P(\text{Class 1}|\text{features}) = 0.0410/0.2993 = 0.1370$ (13.70%)

$P(\text{Class 2}|\text{features}) = 0.1857/0.2993 = 0.6204$ (62.04%)

$P(\text{Class 3}|\text{features}) = 0.0726/0.2993 = 0.2426$ (24.26%)

Choose class with highest probability: Class 2 (62.04%)

Produce the confusion matrix for predicting the credit rating using Naive Bayes on the test set, and also report the overall accuracy rate.

Make predictions on test set

```

nb.test.pred <- predict(credit.nb, newdata = credit.test)
# Create confusion matrix
nb.test.cm <- table(truth = credit.test$credit.rating, prediction = nb.test.pred)
      prediction
truth 1  2  3
1    39 48  9
2    23 136 24
3    18  62 34
nb.test.accuracy <- sum(diag(nb.test.cm)) / sum(nb.test.cm)
0.5318066

```

Based on the confusion matrices reported in the preceding parts,
 # Which of the classifiers look to be the best? Which look to be the worst? Are there any categories that all classifiers seem to have trouble with?

Print accuracy of all models

```

print(paste("Decision Tree Accuracy:", test.accuracy))
"Decision Tree Accuracy: 0.56234096692112"
print(paste("Random Forest Accuracy:", rf.accuracy))
"Random Forest Accuracy: 0.5801527"
print(paste("Default SVM Accuracy:", svm.accuracy))
"Default SVM Accuracy: 0.506361323155216"
print(paste("Tuned SVM Accuracy:", svm.tuned.accuracy))
"Tuned SVM Accuracy: 0.732824427480916"
print(paste("Naive Bayes Accuracy:", nb.test.accuracy))
"Naive Bayes Accuracy: 0.531806615776081"

```

Explanation

1. Best Performing Classifier:

The Tuned SVM clearly outperforms all other classifiers with an accuracy of 73.28%

This significant improvement over the default SVM (50.64%) shows the importance of proper parameter tuning

The tuned parameters (gamma = 0.01, cost = 0.1) helped achieve better generalization

2. Second Best Performer:

Random Forest comes in second with 58.02% accuracy

This suggests that ensemble methods were able to capture some of the complex relationships in the data better than single decision trees

3. Middle Range Performers:

Decision Tree: 56.23%

Naive Bayes: 53.18%

These classifiers performed moderately, with accuracies in the mid-50% range

4. Worst Performing Classifier:

Default SVM with 50.64% accuracy

This poor performance before tuning suggests the default parameters were not suitable for this specific classification problem

5. Common Challenges Across Classifiers:

5.1. Class 3 (High Risk) Prediction:

Looking at the confusion matrices, all classifiers struggled most with correctly identifying Class 3 customers

There is a consistent pattern of misclassifying Class 3 as Class 2

This suggests that the distinction between medium and high-risk customers is particularly challenging

5.2. Class Imbalance:

The models generally performed better on Class 2 (medium risk), which was the majority class

This is evident in the confusion matrices where there is a bias towards predicting Class 2

This suggests that class imbalance might be affecting the model performance

```
###
```

```
# Consider a simpler problem of predicting whether a customer gets a credit rating of A or not.
```

```
# Fit a logistic regression model to predict whether a customer gets a credit rating of A using all of the other variables in the dataset, with no interactions.
```

```
# Create a binary target variable (1 for credit rating "A", 0 otherwise)
```

```
credit.train$rating_A <- ifelse(credit.train$credit.rating == "1", 1, 0)
```

```
credit.test$rating_A <- ifelse(credit.test$credit.rating == "1", 1, 0)
```

```
# Fit logistic regression model
```

```
set.seed(123)
```

```
credit.logit <- glm(rating_A ~ . - credit.rating,
```

```
data = credit.train,
```

```
family = binomial("logit"))
```

```
# Make predictions on test set
```

```
logit.prob <- predict(credit.logit, newdata = credit.test, type = "response")
```

```
logit.pred <- ifelse(logit.prob > 0.5, 1, 0)
```

```
# Create confusion matrix
```

```
logit.cm <- table(Actual = credit.test$rating_A, Predicted = logit.pred)
```

```
  Predicted
```

```
Actual 0 1
```

```
0 263 34
```

```
1 62 34
```

```
# Calculate accuracy
```

```
logit.accuracy <- sum(diag(logit.cm)) / sum(logit.cm)
```

```
0.7557252
```

```
# Report the summary table of the logistic regression model fit.
```

```
summary(credit.logit)
```

```
#Call:
```

```
#glm(formula = rating_A ~ . - credit.rating, family = binomial("logit"), data = credit.train)
```

```
#Coefficients: (2 not defined because of singularities)
```

```
#Estimate Std. Error z value Pr(>|z|)
```

```
##(Intercept) -7.596e+00 1.539e+00 -4.935 8.03e-07 ***
```

```

#functionary1                2.233e+00 1.635e-01 13.655 < 2e-16 ***
#re.balanced..paid.back..a.recently.overdrawn.current.acount1 2.076e+00 6.264e-01
3.314 0.000918 ***
#FI3O.credit.score1          3.770e+00 1.040e+00 3.624 0.000290 ***
#credit.refused.in.past.1    -1.545e+00 3.414e-01 -4.523 6.08e-06 ***
#avrg..account.balance.12.months.ago4 -6.924e-01 2.525e-01 -2.743
0.006092 **
#avrg..account.balance.11.months.ago2 -5.656e-01 2.498e-01 -2.264
0.023559 *
#avrg..account.balance.8.months.ago4 -6.562e-01 2.523e-01 -2.601
0.009293 **
#avrg..account.balance.7.months.ago2  5.059e-01 2.520e-01 2.008
0.044673 *
#max..account.balance.3.months.ago5   -5.654e-01 2.406e-01 -2.350
0.018775 *
#min..account.balance.1.months.ago2    5.237e-01 2.387e-01 2.194
0.028251 *
#Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#(Dispersion parameter for binomial family taken to be 1)
#Null deviance: 1753.0 on 1568 degrees of freedom
#Residual deviance: 1220.6 on 1407 degrees of freedom
#AIC: 1544.6
#Number of Fisher Scoring iterations: 16
Key Significant Predictors:
1. Strongly Significant (p < 0.001):
functionary1 (2.233, p < 2e-16): Being a functionary strongly increases the odds of getting
credit rating A
re-balanced account1 (2.076, p < 0.001): Having re-balanced an overdrawn account
increases the odds of rating A
FI3O credit score1 (3.770, p < 0.001): Having a FI3O credit score dramatically increases the
odds of rating A
credit refused in past1 (-1.545, p < 0.001): Being previously refused credit significantly
decreases the odds of rating A

2. Moderately Significant (p < 0.01):
avrg. account balance 12 months ago4 (-0.692, p < 0.01): Negative effect
avrg. account balance 8 months ago4 (-0.656, p < 0.01): Negative effect

3. Mildly Significant (p < 0.05):
Several account balance variables show mild significance

- Customer history factors (functionary status, credit score, previous refusals) have the
strongest impact
- Recent account activity (particularly minimum balances in recent months) matters
- The pattern of significance across different time periods suggests that both long-term
history and recent activity influence credit rating decisions

```

Which predictors of credit rating appear to be significant at 5% significance level? How do you determine the predictors are significant at 5% significance level.

A predictor is statistically significant at the 5% level when its p-value (shown in the "Pr(>|z|)" column) < 0.05 in the output.

Significant Predictors in the Model:

1. Customer Characteristics:

functionary1 (p < 2e-16)

re-balanced account1 (p = 0.000918)

FI3O credit score1 (p = 0.000290)

credit refused in past1 (p = 6.08e-06)

2. Account Balance Variables:

avrg. account balance 12 months ago4 (p = 0.006092)

avrg. account balance 11 months ago2 (p = 0.023559)

min. account balance 10 months ago4 (p = 0.027513)

min. account balance 10 months ago5 (p = 0.035004)

avrg. account balance 8 months ago4 (p = 0.009293)

avrg. account balance 7 months ago2 (p = 0.044673)

max. account balance 3 months ago5 (p = 0.018775)

min. account balance 2 months ago2 (p = 0.031906)

min. account balance 1 months ago2 (p = 0.028251)

min. account balance 1 months ago4 (p = 0.040905)

Fit an SVM model of your choice to the training set.

Pick 1 customer from the training set and fit a svm model to predict rating for that customer using 4 most significant variables got from the logistic regression

The 4 most significant variables from logistic regression

```
significant.vars <- c("functionary",  
                    "re.balanced..paid.back..a.recently.overdrawn.current.acount",  
                    "FI3O.credit.score",  
                    "credit.refused.in.past.")
```

Pick one random customer from training set

```
set.seed(456)
```

```
customer.index <- sample(1:nrow(credit.train), 1)
```

```
selected.customer <- credit.train[customer.index, ]
```

```
actual.rating <- selected.customer$credit.rating #actual rating: 2
```

Create a simplified dataset with only the significant variables

```
train.simplified <- credit.train[, c(significant.vars, "credit.rating")]
```

```
customer.simplified <- selected.customer[, significant.vars, drop=FALSE]
```

Fit SVM model using only these 4 significant variables

```
set.seed(123)
```

```
svm.simplified <- svm(credit.rating ~ .,  
                    data = train.simplified,  
                    kernel = "radial",  
                    gamma = 0.01,  
                    cost = 10)
```

Predict the rating for the selected customer

```
svm.simplified.pred <- predict(svm.simplified, customer.simplified) # prediction: 2
```

```
print(paste("Predicted rating for the selected customer: ", svm.simplified.pred))
"Predicted rating for the selected customer: 2"
```

Produce an ROC chart comparing the logistic regression and the SVM results of predicting the test set. Comment on any differences in their performance.

Load required library

```
install.packages('ROCR')
```

Load required library

```
library(ROCR)
```

Get probabilities for both models

For logistic regression (probability of class A)

```
logit.prob <- predict(credit.logit, newdata = credit.test, type = "response")
```

For SVM (probability of class A)

```
svm.prob <- predict(credit.svm.tuned, newdata = credit.test, probability = TRUE)
```

```
svm.prob <- attr(svm.prob, "probabilities")[,"1"] # Extract probabilities for class 1 (A)
```

Create prediction objects for ROC curves

```
logit.pred <- prediction(logit.prob, credit.test$rating_A)
```

```
svm.pred <- prediction(svm.prob, credit.test$rating_A)
```

Calculate ROC curves

```
logit.roc <- performance(logit.pred, "tpr", "fpr")
```

```
svm.roc <- performance(svm.pred, "tpr", "fpr")
```

Calculate AUC values

```
logit.auc <- performance(logit.pred, "auc")@y.values[[1]]
```

```
svm.auc <- performance(svm.pred, "auc")@y.values[[1]]
```

Create ROC plot

```
plot(logit.roc, col = "blue", main = "ROC Curves: Logistic Regression vs SVM", lwd = 2)
```

```
plot(svm.roc, col = "red", add = TRUE, lwd = 2)
```

Add reference line (random classifier)

```
abline(a = 0, b = 1, lty = 2, col = "gray")
```

1. ROC Curves:

Both models show ROC curves above the diagonal reference line, indicating better-than-random performance

The curves show the tradeoff between True Positive Rate (sensitivity) and False Positive Rate (1-specificity) at various classification thresholds

2. AUC (Area Under Curve):

The logistic regression models AUC value shows it has good discriminative ability

The tuned SVM model is AUC demonstrates strong performance in distinguishing between class A and non-A ratings

3. Model Comparison:

3.1. Strengths of Logistic Regression:

More interpretable results with clear coefficient values showing feature importance
The significant predictors identified (functionary, re-balanced account, FICO credit score, credit refused in past) provide clear business insights
Better suited for understanding probability estimates
Higher overall accuracy

3.2. Strengths of SVM:

Higher overall accuracy (73.28% vs logistic regression)
Better handling of non-linear relationships in the data
More robust to outliers
Particularly effective after parameter tuning

4. Model Complexity:

Logistic regression provides a linear decision boundary
SVM with radial kernel can capture more complex, non-linear relationships